

BLM1031 Structural Programming – Hw-2

SUBMISSION INFORMATION

The project must be submitted to the project module defined in the online.yildiz system by **Friday, May 31, 2026, at 23:59**, according to the rules below.

Submission format:

StudentID.zip

(Inside the ZIP file, there must be only StudentID.c source code file and StudentID.pdf report.)

Uploading project links from cloud storage systems such as Google Drive, OneDrive, etc. to the online.yildiz.edu.tr system will **strictly not be accepted**. Only direct file uploads are allowed.

Late submissions cannot be made after the system closes. In addition, submission via email or any other method is not possible. Please keep a screenshot showing that your submission to the system was completed successfully as proof. Objections and excuses without proof will not be considered.

IMPORTANT RULES FOR PROGRAM AND REPORT DESIGN

First, read the general coding rules in the document below:

https://docs.google.com/document/d/1ySbmyym0s3VGGVW_Zm5vN7o7N0ttxdgPZ4yorHt5z7E/edit?usp=sharing

Subject: In this project, a small-scale, file-processing-based LIBRARY AUTOMATION application based on the working logic of relational databases will be implemented.

Functional Requirements

1- Author Add/Delete/Edit/List Operations:

Each author's name, surname, and an automatically incremented authorID value (1, 2, ..., N) assigned for every new author record must be stored in a SINGLY LINKED LIST sorted according to the authorID value and kept in a file named authors.csv. The information entered by the user for authors and stored in the file must be editable or deletable when requested, and the updated version must always be maintained in the same file.

2- Student Add/Delete/Edit/List Operations:

Each student must have a name, surname, a library score initially set to 100, and a unique 8-digit student number. Student information must be stored both in a dynamically allocated array defined using a struct within the application and in a file named Students.csv. The stored information must be editable or deletable when requested, and the updated version must always be maintained in the same file.

3- Book Add/Delete/Edit/List Operations: Each book must have a title, a 13-digit ISBN number, and quantity information. Since multiple copies of the same book may exist in the library, each copy must also be assigned a unique tag number during registration in the format ISBNX_1, ISBNX_2, ..., ISBNX_N by automatically appending a number to the corresponding ISBN value. Books must first be stored according to their names and then according to their ISBN numbers using a SINGLY LINKED LIST structure as shown below. Additionally, a DYNAMIC ARRAY must be created for each book according to the number of copies, and these lists must store the book's TAG number and borrowing status. In this context, if a book has been borrowed, the relevant student's ID information must be stored to indicate which student borrowed the book. If the book has not been borrowed, the text "ON SHELF" must be stored in this field. The important point here is designing the necessary structure to store either the studentID or the text "ON SHELF." The relationship between each BOOK and its copies must be modeled using SINGLY LINKED LISTS as shown in Figure-1, and this data must also be stored consistently in CSV-type files. A CSV file is a text file in which the data belonging to each record is separated by commas (,). Example: Java Programming, 1234567891011, 5

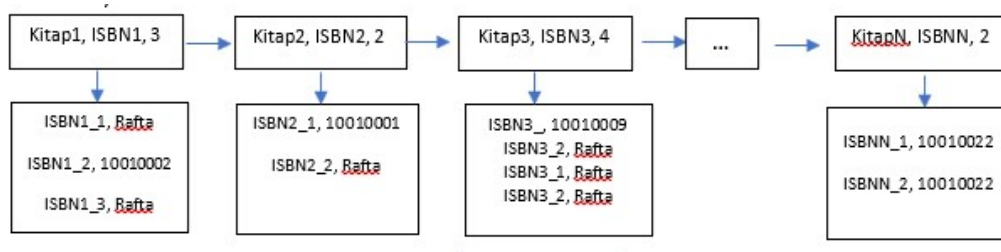


Figure 1 Relationships between books and their copies or samples

4- Book–Author Association Operations: Considering that a book may be written by multiple authors and an author may write many books, a BookISBN – AuthorID matching structure must be implemented. These matching records must be stored in the BookAuthor.csv file as shown in Figure-2. A struct must be designed for the data to be read from this file. After allocating space equal to the number of rows (N) in the file, the data must be stored in a dynamic STRUCT ARRAY of size N as long as the program remains open. According to the example in Figure-2, the book with code 1234567891011 was written by authors with IDs 1, 2, and 3, while the book with code 1234567891012 was written only by the author with ID 1. As understood from the example, an author may write multiple books, and a book may have multiple authors. Only for the book-author matching operation, a random-access file format may be used. A text file format like the one shown in Figure-2 must not be used. The use of .csv files is mandatory for all other items.

1234567891011,1	1234567891011,-1
1234567891011,2	1234567891011,2
1234567891011,3	1234567891011,3
1234567891012,1	1234567891012,-1
1234567891013,4	1234567891013,4
1234567891014,4	1234567891014,4
1234567891014,1	1234567891014,-1

Figure 2 Book–Author Relationship/Mapping

NOTE 1: When an author record entered through Item-1 is deleted from the system, the related file below must also be updated. Since the related author will no longer exist, the value -1 must be written in this field.

NOTE 2: During Book–Author association operations, no action should be performed for unregistered authors or books, and a warning message must be displayed. All necessary checks must be performed to ensure data consistency.

5- Student Book Borrowing Operations:

A student may borrow multiple books, and a book may be borrowed by multiple students at different times. Therefore, this information must be stored consistently. Design a CSV file and a compatible struct to store the borrowing and return status of each book copy. In this data structure, BookTagNo, StudentID, Transaction Type, and Date information must be stored as shown in Figure-3. The transaction type information must be coded as:

- BOOK BORROWING – 0
- BOOK RETURN – 1

In addition, when a book is borrowed, the Shelf Status Information of the book must be updated in the related files and data structures as “On Shelf” or “StudentNo” as specified in Figure-1 and Item-3.

NOTE 1: During book borrowing and return operations, no action should be performed for unregistered students or books, and a warning message must be displayed. Necessary checks must be performed to ensure data consistency and integrity. Additionally, if a student’s score becomes negative, the transaction must be canceled with a “Book cannot be issued” warning.

NOTE 2: If a book is returned more than 15 days after the borrowing date, the related student must receive a penalty of -10 points, and this information must be updated in the related file and data structure.

NOTE 3: During book borrowing operations, if all copies of a book are already borrowed by other students, the warning “TRANSACTION FAILED” must be displayed.

```
1234567891011_1,10011088,0,12.10.2022
1234567891018_2,10011048,0,12.10.2022
1234567891011_1,10011088,1,22.10.2022
1234567891012_1,10011048,1,27.10.2022
1234567891018_2,10011088,0,28.10.2022
```

Figure 1 Book Borrowing

MENUS REQUIRED IN THE PROGRAM

STUDENT OPERATIONS:

1. **Add, Delete, Update Student:**
All student-related information must be obtained from the user, and Add/Delete/Update operations must be performed separately on files and dynamic arrays.
2. **Display Student Information:**
The personal information (Name, Surname, ID, Score, etc.) and all book transaction records of the student whose ID or Name-Surname information is entered must be listed.
3. **List Students Who Have Not Returned Books**
4. **List Penalized Students**
5. **List All Students**
6. **Borrow/Return Book**

BOOK OPERATIONS:

7. **Add, Delete, Update Book:**
All book-related information must be obtained from the user, and Add/Delete/Update operations must be performed separately on files and linked lists and dynamic arrays.
8. **Display Book Information:**
According to the Book Name entered by the user, all information about the book and its sample copies must be listed.
9. **List Books on the Shelf**
10. **List Books Not Returned on Time**
11. **Match Book–Author:**
The related file and struct array must be updated.
12. **Update the Author of a Book:**
The related file and struct array must be updated.

AUTHOR OPERATIONS:

13. **Add, Delete, Update Author:**
All author-related information must be obtained from the user, and Add/Delete/Update operations must be performed separately on files and linked lists.
14. **Display Author Information:**
According to the Author Name entered by the user, all information about the author and all books belonging to that author must be listed.

Coding Requirements

- Design the menu and the necessary functions for each operation specified above. Use function pointers to avoid repetition in functions operating with similar logic.
- Functions must be defined for all add, delete, and update operations.
- After performing data modification operations such as Add, Delete, and Update on the related data structure, the data must be saved to the file.
- The use of static and global variables in the program is prohibited.
- All memory allocation operations must be performed using dynamic memory management functions.
- Appropriate structure definitions must be created for each object in the files described above.

- Unless otherwise specified, the files to be created must be in CSV file format.
- Unnecessary or redundant member variables must not be used in struct designs. For example, if a book is borrowed, either the borrower's ID value or the text "ON SHELF" must be stored. Appropriate struct/union/enum definitions must be designed for such situations.

Project Reporting

- In your report, demonstrate that each function you wrote works correctly by entering a sufficient number of student and course records to test them.
- For this purpose, when running the program, copy everything displayed on the screen and entered by the user from the command line and include it in the PDF report file.
- When the program terminates, include the contents of all files storing the data in your report.
- Add the references you used and what this project contributed to you at the end of the report.

SUBMISSION REQUIREMENTS

Inside the STUDENTNUMBER.RAR/ZIP file:

Example: 10011089.rar or 10011089.zip

The following must be included:

- StudentNumber.pdf report file
- StudentNumber.c source code
- All files generated as a result of the test execution

NOTE:

The report must include all outputs, including the table of contents, contact information, and similar details.

FOR GENERAL CODING RULES:

<https://docs.google.com/document/d/e/2PACX-1vSrFZcy-azGnsbvZlIrvvgq6OPlhDH0eAuzmoUMwJgxLzfNdHtQZ9AD1xaeA1H35jeVeSHle11u-B00/pub>

All assignments will be checked using similarity detection software. Assignments determined to be similar will be considered plagiarism, and each related assignment will receive a grade of 0 (zero).

NOTE:

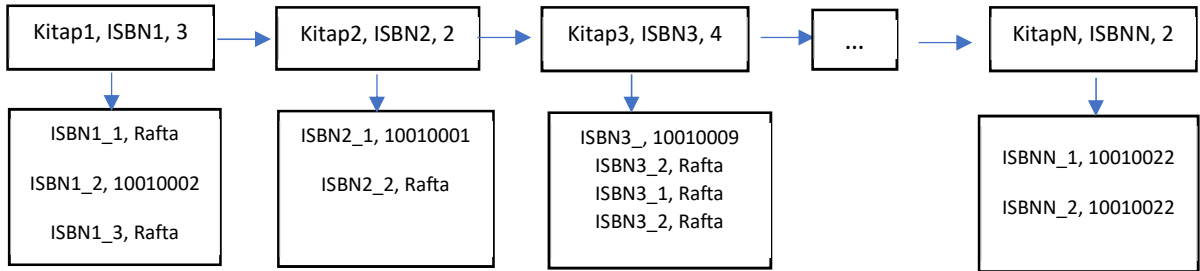
Upload your project via online.yildiz.edu.tr before the final submission deadline. Since the system will close automatically, late submissions via e-mail or other methods will not be accepted. Save a screenshot showing that the upload was successful as proof. Assignments sent later via e-mail without proof will not be accepted.

Good Luck.

Konu: Bu projede ilişkisel veri tabanlarının çalışma mantığına dayanan, küçük ölçekli ve dosya işlemleri tabanlı KÜTÜPHANE OTOMASYONU uygulaması gerçekleştirilecektir.

İşlevsel Gereksinimler:

- Yazar Ekleme/Silme/Düzenleme/Listeleme İşlemleri:** Her bir yazarın adı, soyadı, her yeni yazar kaydında otomatik artarak (1, 2, ..., N) verilecek yazarID bilgisi TEK YÖNLÜ LINKED LISTEDE yazarID değerine göre sıralı bir şekilde saklanmalı **yazarlar.csv** isimde dosyada tutulmalıdır. Yazarlar için kullanıcıdan alınıp dosyada saklanan bu bilgiler istenirse silinebilmeli veya düzeltilebilmelidir ve son hali aynı dosyada güncel olarak bulunmalıdır.
- Öğrenci Ekleme/Silme/Düzenleme/Listeleme İşlemleri:** Her bir öğrencinin adı, soyadı, başlangıçta 100 olan kütüphane puanı ve her bir öğrenci için benzersiz olan 8 haneli öğrenci numarası olmalıdır. Öğrencilere ait bilgiler dosyada ve uygulama içerisinde tanımlanacak struct yapısında **dinamik bir dizide** ve **Oğrenciler.csv** isimli dosyada saklanmalıdır. Saklanan bu bilgiler istenirse silinebilmeli veya düzeltilebilmelidir ve son hali aynı dosyada güncel olarak bulunmalıdır.
- Kitap Ekleme/Silme/Düzenleme/Listeleme İşlemleri:** Her bir kitabın adı, 13 Haneli ISBN numarası ve adet bilgisi olmalıdır. Bir kitaptan birden fazla sayıda örnek kütüphanede olabileceği için, kayıt oluşturma sırasında her bir örnek kitaba ayrıca benzersiz olan etiket numarası ISBN numarasına ISBN_1, ISBN_2, ISBN_N şeklinde otomatik eklenecek sayı ile verilmelidir. Kitaplar önce isimlerine sonra da ISBN numaralarına göre TEK YÖNLÜ LINKED LIST ile aşağıdaki gibi saklanmalıdır. Her kitabın örnek sayısı kadar kitabın bilgisi için ayrıca **DİNAMİK BİR DİZİ** oluşturulmalı ve bu listelerde kitabın ETİKET numarası ile kitabın ödünç alınma durumu saklanmalıdır. Bu bağlamda, kitap ödünç alınmış ise hangi öğrenci tarafından ödünç alındığını göstermek için ilgili öğrencinin ID bilgisi yer almalıdır. Eğer kitap ödünç alınmamışsa, bu alanda RAFTA bilgisi yazmalıdır, burada dikkat edilmesi gereken husus öğrenciID ya da RAFTA metninin saklanması için gerekli yapının kurgulanmasıdır. Her bir KİTAP ve bunların örnekleri arasındaki ilişki Şekil-1’de gösterildiği gibi TEK YÖNLÜ LINKED LIST’ler kullanılarak modellenmeli, ayrıca bu veriler uyumlu bir şekilde CSV türündeki dosyalarda saklanmalıdır. CSV dosyası; her bir örneğe ait verinin virgül (,) işareti ile ayrıldığı metin dosyasıdır. Örnek: Java Programlama, 1234567891011, 5



Şekil 2 Kitap ve Örnekleri Arasındaki İlişki

- Kitap – Yazar İlişkilendirme İşlemleri:** Bir kitabın birden çok yazar tarafından yazılma, bir yazarın da çok sayıda kitap yazabilme durumu dikkate alınarak, **KitapISBN - YazarID** eşleştirmesi yapılmalıdır. Bu eşleştirme bilgileri **KitapYazar.csv** dosyasında Şekil-2’de gösterildiği gibi saklanmalıdır. Bu dosya üzerinde yapılacak okuma işlemlerinde kaydedilecek veriler için bir struct tasarlanmalıdır. Bu struct yoluyla dosyadaki satır sayısı (N) kadar yer açıldıktan sonra, N elemanlı dinamik STRUCT DİZİSİNDE veriler program açık kaldığı sürece saklanmalıdır. Şekil-2’de bulunan örneğe göre; 1234567891011 kodlu kitap 1, 2 ve 3 ID’ye sahip yazarlar tarafından yazılmış olup, 1234567891012 kodlu kitabı ise sadece 1 ID’ye sahip yazar yazmıştır. Örnekten de anlaşıldığı gibi bir yazar çok sayıda kitap yazabilirken, bir kitap çok sayıda yazar tarafından yazılabilir. Sadece kitap-yazar eşleştirmesi için rastgele erişilebilir dosya formatı kullanabilirsiniz. Şekil-2deki gibi text dosyası yapılmamalıdır. **Diğer maddeler için .csv dosyası kullanımı zorunludur.**

NOT1: Madde-1 ile girişi yapılan yazar bilgisi sistemden silindiğinde aşağıdaki dosyada güncelleme yapılmalıdır. İlgili yazar artık olmayacağı için bu alanda -1 yazmalıdır.

NOT2: Kitap – Yazar eşleştirme işlemlerinde kayıtlı olmayan bir Yazar veya kitap için işlem yapılmamalı ve uyarı vermelidir. Veri tutarlılığı adına gerekli tüm kontroller yapılmalıdır.

1234567891011,1 1234567891011,2 1234567891011,3 1234567891012,1 1234567891013,4 1234567891014,4 1234567891014,1	1234567891011,-1 1234567891011,2 1234567891011,3 1234567891012,-1 1234567891013,4 1234567891014,4 1234567891014,-1
---	--

Şekil 3 Kitap Yazar Eşleştirmesi (Sol: Yazar silinmeden önce, Sağ: Yazar silindikten sonra)

- 5- **Öğrenci Kitap Ödünç Alma İşlemleri:** Bir öğrenci çok sayıda kitabı ödünç alabilirken, bir kitap farklı zamanlarda çok sayıda öğrenci tarafından da ödünç alınabilir. Bu nedenle bu bilgilerin tutarlı bir şekilde saklanması gerekmektedir. Her kitaba ait örneğin ödünç alınma ve teslim edilme durumlarını saklayan bir CSV dosyası ve buna uyumlu olacak bir struct tasarlayınız. Bu veri yapısında ÖğrenciID, KitapEtiketNO, İşlem Türü ve Tarih Bilgisi Şekil-3 ile gösterildiği gibi saklanmalıdır. İşlem türü bilgisi; **kitabın ÖDÜNÇ ALINMASI – 0** ya da **kitabın TESLİM EDİLMESİ – 1** olarak kodlanmalıdır. Bununla birlikte kitap ödünç alındığında Şekil-1 ve Madde-3 ile belirtildiği gibi, bir kitabın Raf Durum Bilgisi, **Rafta ya da ÖğrenciNo** olarak ilgili dosyalarda ve veri yapısında güncellenmelidir.

NOT1: Kitap ödünç alma ve teslim işlemlerinde kayıtlı olmayan bir öğrenci veya kitap için işlem yapılmamalı ve uyarı vermelidir. Veri tutarlılığı ve bütünlüğü için gerekli kontroller yapılmalıdır. Ayrıca, bir öğrencinin Puan Bilgisi negatif durumda olursa kitap verilemez uyarısı ile işlem iptal edilmelidir.

NOT2: Kitap teslimi kitabın ödünç alındığı tarihten 15 günden sonra yapılırsa ilgili öğrenciye -10 ceza puanı verilmeli ve bu bilgiler ilgili dosya ve veri yapısında güncellenmelidir.

NOT3: Kitap ödünç alma işlemleri sırasında, bir kitaba ait tüm örnekler başka öğrenciler tarafından ödünç alınmışsa İŞLEM BAŞARISIZ uyarısı verilmelidir.

```
1234567891011_1,10011088,0,12.10.2022
1234567891018_2,10011048,0,12.10.2022
1234567891011_1,10011088,1,22.10.2022
1234567891012_1,10011048,1,27.10.2022
1234567891018_2,10011088,0,28.10.2022
```

Şekil 4 Kitap Ödünç Alma İşlemleri

PROGRAMDA İSTENEN MENÜLER

ÖĞRENCİ İŞLEMLERİ:

- 1- **Öğrenci Ekle, Sil, Güncelle:** Öğrenciye ait tüm bilgilerin kullanıcıdan alınıp, dosyalar ve dinamik dizi üzerinde **Ekle/Sil/Güncelle** işlemlerinin ayrı ayrı gerçekleştirilmesi gerekmektedir.
- 2- **Öğrenci Bilgisi Görüntüleme:** ID bilgisi veya Ad-Soyad bilgisi alınan öğrencinin kişisel bilgileri (Ad, Soyad, ID, Puan vb) ve tüm kitap hareketleri listelenmelidir.
- 3- **Kitap Teslim Etmemiş Öğrencileri Listele:**
- 4- **Cezalı Öğrencileri Listele**
- 5- **Tüm Öğrencileri Listele**
- 6- **Kitap Ödünç Al/Teslim Et**

KİTAP İŞLEMLERİ:

- 7- **Kitap Ekle, Sil, Güncelle:** Kitaplara ait tüm bilgilerin kullanıcıdan alınıp, dosyalar ve linkli liste ve dizi üzerinde **Ekle/Sil/Güncelle** işlemlerinin ayrı ayrı gerçekleştirilmesi gerekmektedir.
- 8- **Kitap Bilgisi Görüntüleme:** Kullanıcıdan alınan Kitap Adı bilgisine göre her bir kitabın ve bu kitabın örnek kopyalarına **ait tüm bilgilerin listelenmesi** işlemleri yapılmalıdır.
- 9- **Raftaki Kitapları Listele:**
- 10- **Zamanında Teslim Edilmeyen Kitapları Listele:**
- 11- **Kitap-Yazar Eşleştir :** İlgili dosya ve struct dizisi üzerinde güncelleme yapılmalıdır
- 12- **Kitabın Yazarını Güncelle:** İlgili dosya ve struct dizisi üzerinde güncelleme yapılmalıdır

YAZAR İŞLEMLERİ:

- 13- **Yazar Ekle, Sil, Güncelle:** Yazarlara ait tüm bilgilerin kullanıcıdan alınıp, dosyalar ve linkli listeler üzerinde **Ekle/Sil/Güncelle** işlemlerinin ayrı ayrı gerçekleştirilmesi gerekmektedir.
- 14- **Yazar Bilgisi Görüntüleme:** Kullanıcıdan alınan Yazar Adı bilgisine göre her bir yazarın bilgilerinin ve bu yazara ait kitaplara **ait tüm bilgilerin listelenmesi** işlemleri yapılmalıdır.

Kodlama Gereklilikleri:

- ❖ Yukarıdaki maddelerde verilen bir menü ve her bir işlem için gerekli fonksiyonları tasarlayınız. **Benzer mantıkla çalışan fonksiyonlarda tekrardan kaçınmak için fonksiyon pointer kullanınız.**
- ❖ Yapılması gereken tüm ekle, sil ve güncelleme işlemler için fonksiyon tanımlamanız gerekmektedir.
- ❖ Ekle, Sil, Güncelle gibi veri üzerinde değişiklik yapılan işlemleri ilgili veri yapısında yaptıktan sonra dosyaya mutlaka kaydediniz.
- ❖ Programlarınızda static ve global değişken kullanımı yasaktır.
- ❖ Her türlü bellek tahsisi, dinamik bellek yönetimi fonksiyonları ile yapılmalıdır.
- ❖ Yukarıda detayları verilen her bir dosyadaki nesne için uygun structure tanımlamalarını yapınız.
- ❖ Oluşturulacak dosyalar aksi belirtilmedikçe CSV dosyası formatında olmalıdır.
- ❖ Struct tasarımlarında gereksiz ve fazladan üye değişken kullanılmamalıdır. Örneğin bir kitap ödünç alınmışsa ödünç alan kişinin ID değeri ya da ödünç alınmamışsa RAFTA bilgisi saklanmalıdır. Bu tür bir durum için gerekli struct/union/enum tanımlamaları yapılmalıdır.

Projenin Raporlanması:

- ❖ Raporunuzda, yazdığınız her bir fonksiyonu test etmek için yeterli sayıda öğrenci ve ders bilgisi girerek yazdığınız fonksiyonların doğru çalıştığını gösteriniz.
- ❖ Bu amaçla programı çalıştırdığınızda ekrana çıkan ve kullanıcının ekran üzerinden giriş yaptığı her şeyi komut satırından kopyalayıp raporunuzu oluşturacak PDF dosyasına ekleyiniz.
- ❖ Program sona erdiğinde verilerin saklandığı tüm dosyaların içeriklerini de raporunuza ekleyiniz.
- ❖ Yararlandığınız kaynakları ve bu projenin size kazandırdıklarını raporun sonuna ekleyiniz

TESLİM EDİLECEKLER:

ÖĞRENCİNO.RAR/ZIP Dosyası içinde: Örn: 10011089.rar veya 10011089.zip

*** ÖğrenciNo.pdf rapor dosyası,

*** ÖğrenciNo.c kaynak kodu ve

*** Test koşumu sonucunda ortaya çıkan tüm dosyaları bulunmalıdır.

NOT: Raporda içindekiler tablosu, iletişim bilgileri vb detaylar dahil tüm çıktıları içermelidir.

KODLAMA HAKKINDA GENEL KURALLAR İÇİN:

[https://docs.google.com/document/d/e/2PACX-1vSrFZcy-](https://docs.google.com/document/d/e/2PACX-1vSrFZcy-azGnsbvZllrzvgq6OPlhDH0eAuzmoUMwJgxLzfNdHtQZ9AD1xaeA1H35jeVeSHle11u-B00/pub)

[azGnsbvZllrzvgq6OPlhDH0eAuzmoUMwJgxLzfNdHtQZ9AD1xaeA1H35jeVeSHle11u-B00/pub](https://docs.google.com/document/d/e/2PACX-1vSrFZcy-azGnsbvZllrzvgq6OPlhDH0eAuzmoUMwJgxLzfNdHtQZ9AD1xaeA1H35jeVeSHle11u-B00/pub)

Tüm ödevler benzerlik denetim programından geçirilecek, benzer olduğu anlaşılan ödevler kopya olarak değerlendirilecek ve ilgili her ödev 0 (sıfır) notu verilecektir.

NOT: Projenizi son teslim tarihine kadar online.yildiz.edu.tr üzerinden yükleyiniz. Sistem otomatik kapanacağı için geç teslimler mail vb yoluyla mümkün değildir. Projenin yüklenmesinin başarılı olduğunu gösteren ekran görüntüsünü kanıt olarak kaydediniz. Kanıt olmaksızın sonradan e-posta yoluyla gönderilen ödevler kabul edilmeyecektir.

Başarılar Dileriz.